

ファイル構成・依存関係マップ

本書は「どの画面が何をするか」「どのファイルがどのファイルを読んでいるか」を一覧できるようにするためのリファレンス。
2026-07-26時点の実装内容に基づく（認証・Claudeコスト管理・利用料アラート・ユーザー名・釣具店検索・AIチャットの実データ
グラウンディング追加後）。仕様の背景・意思決定理由は基本設計書.html／詳細設計.html／DEテーブル設計.htmlを参照。

1. 画面一覧（フロントエンド）

URL	画面名	ページファイル	やること
/	おすすめ	pages/TopPage.tsx	AIスコア順のTOP3・一覧表示、AI分析実行、現在地から新スポット探索、現在地基準の再ランキング
/map	地図	pages/MapPage.tsx	Google Maps上に全スポットをスコア色分けピン表示
/spots	スポット	pages/SpotsPage.tsx	全スポット一覧（スコア・魚種・ナビリンク）
/tackle-shops	釣具店【2026-07-26追加】	pages/TackleShopsPage.tsx	釣具店の検索（現在地・キーワード）。スコアなし、DB保存なし
/favorites	保存済み	pages/FavoritesPage.tsx	お気に入り登録済みスポットのみ表示
/posts	釣果	pages/PostsPage.tsx	釣果投稿の一覧・作成・編集・削除（投稿者の表示名を表示。編集は本人の投稿のみ、投稿先スポットは変更不可【2026-07-26追加】）
/chat	AI相談	pages/ChatPage.tsx	写真添付対応のAIチャット、会話履歴。実データ（Spots/Recommendations）に基づいた回答に対応【2026-07-26】

ルーティング定義は App.tsx。全ページ共通のヘッダーは components/NavBar.tsx（ロゴクリックで / に戻る、モバイルではラベル非表示、右端にログイン状態を表示）。

ログイン不要（閲覧）：おすすめ・地図・スポット・釣果投稿の一覧表示。

ログイン必須（操作）：お気に入り登録/削除、投稿の作成/編集/削除、AI相談、AI分析実行、現在地から新スポット探索、釣具店検索（Places API呼び出しのコスト保護のため）。未ログインでこれら进行操作しようとするとCognito Hosted UI（Googleログイン画面）へ自動遷移するか（お気に入り・投稿作成・AI分析実行・新スポット探索・釣具店検索）、ページ自体がログイン案内を表示する（AI相談）。

TOP3カード（RecommendationCard.tsx）はスポット写真が登録されている場合、カード上部にヒーロー写真を表示する（その他一覧・お気に入り一覧はホバー/長押しプレビューのみ。2026-07-26追加）。

2. フロントエンド ファイル依存関係

2.1 ページ → フック → コンポーネント

ページ	使用するフック	使用する主なコンポーネント
TopPage.tsx	useRecommendations, useFavorites	RecommendationCard, AiBatchButton, SpotDiscoveryButton, DetailModal, NearbyModal
MapPage.tsx	（なし。api/client を直接呼ぶ）	@vis.gl/react-google-maps（外部ライブラリ）

SpotsPage.tsx	(なし。api/client を直接呼ぶ)	ImagePreviewPopover
TackleShopsPage.tsx 【2026-07-26追加】	useGeolocation	(なし。api/client の searchTackleShops を直接呼ぶ)
FavoritesPage.tsx	useFavorites, useRecommendations	RecommendationCard, DetailModal
PostsPage.tsx	usePosts	PostForm
ChatPage.tsx	useChat	ChatInput, ChatHistoryPanel

2.2 コンポーネント間の依存

コンポーネント	使われる場所	内部で使うもの
RecommendationCard.tsx	TopPage, FavoritesPage, NearbyModal	utils/score.ts, ImagePreviewPopover
DetailModal.tsx	TopPage, FavoritesPage, NearbyModal	utils/score.ts, ImagePreviewPopover, api/client (写真アップロード)
NearbyModal.tsx	TopPage	hooks/useGeolocation, utils/distance.ts, utils/score.ts, RecommendationCard
ImagePreviewPopover.tsx	RecommendationCard, SpotsPage, DetailModal	hooks/useLongPress, api/wikipedia.ts
PostForm.tsx	PostsPage	api/client (署名付きアップロード・投稿作成)
ChatInput.tsx / ChatHistoryPanel.tsx	ChatPage	(propsで受けたコールバックのみ。API直呼びなし) ChatInputのみ2026-07-26追加のhooks/useGeolocationで現在地共有トグルを実装
AiBatchButton.tsx / SpotDiscoveryButton.tsx	TopPage	SpotDiscoveryButtonのみ api/client (runSpotDiscovery) を直接呼ぶ
ProfileModal.tsx 【2026-07-26追加】	NavBar	(propsで受けたコールバックのみ。API直呼びなし)

2.3 フック → APIクライアント

フック	呼び出す api/client.ts の関数
useRecommendations.ts	getRecommendations, runAiBatch (未ログイン時はrunAiBatch実行前にsigninRedirect)
useFavorites.ts	getFavorites, addFavorite, removeFavorite (2026-07-26: userId引数を廃止。未ログイン時はfetchせず空配列、トグル時はsigninRedirect)
usePosts.ts	getPosts, createPost, deletePost (削除失敗時はdeleteErrorに反映)
useChat.ts	sendChatMessage, getChatHistory, getChat, deleteChat, getPresignedUploadUrl, uploadImageToS3 (429応答はレート制限メッセージとして表示。2026-07-26: sendChatMessageにlat/lngを渡せるようになった)
useProfile.ts 【2026-07-26追加】	getMyProfile, updateMyProfile (未ログイン時はfetchしない。NavBarから使われる)

useGeolocation.ts	(なし。navigator.geolocation をラップするのみ。 TopPage/SpotDiscoveryButton/ChatInput/TackleShopsPageから使われる)
useLongPress.ts	(なし。hover/長押し検知のみのUIフック)

api/client.ts が全エンドポイント呼び出しを一手に引き受ける唯一の窓口（共有axiosインスタンス）。 api/wikipedia.ts だけは例外で、魚種画像取得のため直接 fetch() でWikipedia REST APIを叩く（バックエンドを経由しない）。

2.5 認証まわりのファイル（2026-07-26追加）

ファイル	役割
auth/authConfig.ts	react-oidc-context の AuthProvider に渡す設定（authority/client_id/redirect_uri）。環境変数 VITE_COGNITO_* から組み立てる。Cognito Hosted UIの独自ログアウトURLを作る cognitoSignOut() もここに定義
auth/AuthTokenSync.tsx	画面には何も描画しない同期用コンポーネント。ログイン状態が変わるたびに api/client.ts の setAuthToken() を呼び、以後のAPIリクエストのAuthorizationヘッダーを最新化する

App.tsx が <AuthProvider> でアプリ全体をラップし、直下に <AuthTokenSync /> をマウントする。各フック・コンポーネントは react-oidc-context の useAuth() を直接呼んでログイン状態を参照する（Reduxのような中央ストアは無く、Contextで完結）。

2.4 純粋ユーティリティ

ファイル	内容	ユニットテスト
utils/score.ts	スコア→色/ラベル変換、距離ベースのスコア近似再計算	utils/score.test.ts
utils/distance.ts	haversine公式による2点間距離計算	utils/distance.test.ts
types/index.ts	Spot/Recommendation/Favorite/Post/Chat等、全体で共有する型定義	-

3. バックエンド ファイル構成（Lambda）

3.1 API系（backend/lambda/api/handlers.py 1ファイルに集約）

関数	エンドポイント	認証	読み書きするテーブル/バケット
getRecommendationsHandler	GET /recommendations	不要	Recommendations（読）, Spots（読、結合用）
getSpotsHandler	GET /spots	不要	Spots（読）
putSpotImageHandler	PUT /spots/{spotId}/image	必須	Spots（更新）
getPostsHandler	GET /posts	不要	Posts（読）, Users（読、投稿者の表示名を結合。2026-07-26追加）
postPostsHandler	POST /posts	必須	Posts（作成。userIdは_get_user_id(event)で取得しリクエストボディの値は無視）

putPostHandler 【2026-07-26追加】	PUT /posts/{postId}	必須	Posts（更新。content/imageUrl/fishCaughtのうち指定されたフィールドのみ更新。所有者チェックはdeletePostHandlerと同じ方式）
deletePostHandler	DELETE /posts/{postId}	必須	Posts（削除。postId単一PKで所有者情報がキーに無いため、削除前にget_itemして自分の投稿か明示チェック。他人の投稿は403）
getFavoritesHandler	GET /favorites	必須	Favorites（読）, Spots（読、結合用）
postFavoritesHandler	POST /favorites	必須	Favorites（作成）
deleteFavoritesHandler	DELETE /favorites/{spotId}	必須	Favorites（削除）
postPresignUploadHandler	POST /uploads/presign	必須	S3 UploadsBucket（署名付きPOSTフォーム発行のみ）
postChatHandler	POST /chat	必須	Chats（読み書き）, Usage（1日あたり利用回数のアトミック加算）, Spots・Recommendations（読、実データグラウンディング用。2026-07-26追加）, S3（画像読み取り）, SSM（Claudeキー）
getChatsHandler	GET /chats	必須	Chats（読、軽量版）
getChatHandler	GET /chats/{chatId}	必須	Chats（読、全メッセージ）
deleteChatHandler	DELETE /chats/{chatId}	必須	Chats（削除）
getMyProfileHandler 【2026-07-26追加】	GET /me	必須	Users（読）。未登録でも404にせずdisplayName:nullで200を返す
putMyProfileHandler 【2026-07-26追加】	PUT /me	必須	Users（作成/更新。表示名は1〜30文字でバリデーション）

2026-07-26追加:「認証必須」のエンドポイントは template.yaml の Api リソースに定義した CognitoAuthorizer（Cognito User Pool Authorizer）を通過して初めて呼び出せる。通過後のイベントには event.requestContext.authorizer.claims.sub にユーザーの一意なID（Cognitoのsub）が入り、_get_user_id(event) ヘルパーがこれを取得する（Favorites/Chatsテーブルは元々userIdがpartition keyの一部のため、他人になりすまして アクセスすること自体ができない設計。PostsテーブルだけpostId単一PKのため上記の通り明示チェックが必要）。

3.2 バッチ系（backend/lambda/batch/）

ファイル	役割	他ファイルとの関係
batch_common.py	DynamoDB/SSM/HTTP GETの共有ヘルパー。 2026-07-26追加: get_user_id/check_and_increment_daily_usage（handlers.pyの同名関数と同じロジックのapi/非依存版、コスト保護レート制限用）	generate_score.py・discover_spots.py・admin_trigger.py・tackle_shops.py から import される（循環import回避のため切り出し）
generate_score.py	generateSpotScoreBatch本体。天気/潮汐取得 →スコア計算→Claude推薦理由生成 →Recommendations保存	batch_common を import。handler冒頭で discover_spots.run_discovery() を呼ぶ（新スポット探索も内包）

discover_spots.py	Google Places APIで新規スポットを探索し Spotsへ追加する中核ロジック（run_discovery）。2026-07-26: Placesの types に"store"を含む候補（釣具店等）を除外するようになった	batch_common を import。generate_score.py から呼ばれる他、discoverSpotsBatch Lambdaとしても単独稼働。tackle_shops.py から search_places/haversine_km が import される
admin_trigger.py	フロントのボタンから対象バッチをEvent(非同期)invokeする汎用ラッパー。2026-07-26: 環境変数 RATE_LIMIT_ACTION/RATE_LIMIT_DAILYでコスト保護のレート制限を追加（上限超過時はバッチを起動せず429）	環境変数 BATCH_FUNCTION_NAME により、同一コードが adminRunAiBatch と adminRunSpotDiscovery の2つのLambdaとして再利用されている（レート制限の上限値もこの環境変数で関数ごとに変えている）
tackle_shops.py 【2026-07-26追加】	釣具店を検索して返すだけのLambda（DB書き込みなし・スコアなし）。DAILY_SEARCH_LIMIT（20件/日）のレート制限あり	discover_spots.py の search_places/haversine_km、batch_common の get_ssm_parameter/get_user_id/check_and_increment_daily_usage を import。同じ../backend/lambda/batch/パッケージに配置

backend/seed_data.py はLambdaではなく、初期スポットデータ投入用の手動実行スクリプト（python seed_data.py）。

3.3 SAM関数 ⇄ ハンドラー対応（infrastructure/template.yaml）

SAM論理ID	Handler指定	トリガー
GenerateSpotScoreBatch	generate_score.handler	EventBridge（毎日AM6:00 JST） ／AdminRunAiBatchFunction経由
DiscoverSpotsBatch	discover_spots.handler	AdminRunSpotDiscoveryFunction 経由のみ（自動スケジュールなし）
AdminRunAiBatchFunction	admin_trigger.handler （BATCH_FUNCTION_NAME=GenerateSpotScoreBatch, RATE_LIMIT_ACTION=ai-batch, RATE_LIMIT_DAILY=3）	POST /admin/run-ai-batch（認証 必須）
AdminRunSpotDiscoveryFunction	admin_trigger.handler （BATCH_FUNCTION_NAME=DiscoverSpotsBatch, RATE_LIMIT_ACTION=spot-discovery, RATE_LIMIT_DAILY=10）	POST /admin/run-spot-discovery （認証必須）
SearchTackleShopsFunction 【2026-07-26追加】	tackle_shops.handler（DAILY_SEARCH_LIMIT=20はコード 内定数）	POST /tackle-shops/search（認証 必須。Places API課金保護のため）

2026-07-26追加（コスト保護）：上記3関数はPlaces/Claude APIの呼び出しを伴うため、AI相談（DAILY_CHAT_LIMIT）と同じ仕組み（UsageTable への ADD によるアトミック加算）で1ユーザー1日あたりの上限を設けている。共有ロジックは batch_common.py の check_and_increment_daily_usage / get_user_id。上限超過時はバッチ起動・検索を 行わず429を返す。

2026-07-26追加: それまでSAMが暗黙生成していたAPI Gatewayを、template.yaml に 明示的な Api（AWS::Serverless::Api）リソースとして定義し直した（Cognitoオーソライザーを使うために必要）。既存のCORS設定もここに統合済み。

4. DynamoDBテーブル一覧

物理名	論理ID	キー構成	書き込み元
fishing-spots	SpotsTable	spotId (PK)	seed_data.py（手動）, discover_spots.run_discovery, putSpotImageHandler
fishing-recommendations	RecommendationsTable	spotId (PK)	generate_score.py（generateSpotScoreBatch実行時のみ）
fishing-favorites	FavoritesTable	userId (PK) + spotId (SK)	postFavoritesHandler / deleteFavoritesHandler
fishing-posts	PostsTable	postId (PK)	postPostsHandler / putPostHandler / deletePostHandler
fishing-chats	ChatsTable	userId (PK) + chatId (SK)	postChatHandler / deleteChatHandler
fishing-usage	UsageTable 【2026-07-26追加】	userId (PK) + dateKey (SK, 例: "chat#2026-07-26")	postChatHandler内の_check_and_increment_daily_usage（ADDによるアトミック加算）。TTL（expiresAt）で翌々日ごろ自動削除
fishing-users	UsersTable 【2026-07-26追加】	userId (PK)	putMyProfileHandler（表示名の作成/更新）。getPostHandlerがscanして投稿者名の結合に使う

S3バケット（fishing-ai-app-uploads-<アカウントID>）はスポット写真・投稿写真・チャット添付画像のアップロード先。書き込みは署名付きPOSTフォーム経由のみ、読み取り（GetObject）のみ公開許可。

5. データの流れ（代表的なシナリオ）

5.1 「AI分析を実行」を押したとき

```
TopPage.tsx（ボタン）
→ useRecommendations.ts の triggerAiBatch()
→ api/client.ts の runAiBatch()
→ POST /admin/run-ai-batch
→ AdminRunAiBatchFunction（admin_trigger.py）
→ GenerateSpotScoreBatch を非同期invoke
→ generate_score.py の handler()
  → discover_spots.run_discovery()（新スポット探索、Spotsへ追加）
  → Spots全件スキャン→天気/潮汐取得→スコア計算→Claude推薦理由生成
  → Recommendationsへ保存
→ フロントは GET /recommendations を定期的にポーリングして完了を検知
```

5.2 チャットで写真付きの質問をしたとき

```
ChatInput.tsx（写真選択 + 送信）
→ useChat.ts の send()
  → api/client.ts の getPresignedUploadUrl() → POST /uploads/presign
  → api/client.ts の uploadImageToS3()（S3へ直接POST、バックエンドを経由しない）
  → api/client.ts の sendChatMessage() → POST /chat
→ postChatHandler（handlers.py）
  → S3から画像バイトを取得 → Claudeへマルチモーダル入力として送信
  → Chatsテーブルへ保存 → 応答を同期的に返却
```

5.3 「近くの釣り場は？」とAI相談で聞いたとき（2026-07-26追加）

```
ChatInput.tsx (現在地共有トグルON + 送信)
→ useChat.ts の send() (lat/lngを含めてsendChatMessage呼び出し)
→ POST /chat
→ postChatHandler (handlers.py)
  → _build_spots_context(lat, lng) がSpots・Recommendationsをscan→結合→
    現在地からの距離 (haversine) で近い順にソート→上位40件をテキスト化
  → _generate_chat_reply() のsystem_instructionに上記テキストを埋め込む
    (「データに無い場所は作り出さない」という指示も併記)
→ Claudeが実データの中から回答を組み立てる
```

6. 使用サービスとコストの目安（2026-07-26追加）

友人にアプリを共有し複数人が使うようになったことを踏まえ、現時点で使っている全サービスと「何に対して課金されるか」「個人～友人数名規模の利用ならどの程度か」を一覧化する。**正確な最新料金は各サービスの公式ページで確認すること**（料金体系は変更されうるため、あくまで目安）。

サービス	課金対象	個人～友人規模での目安
Lambda	呼び出し回数・実行時間	無料枠内（月100万回・40万GB秒まで無料）
API Gateway (REST)	APIコール数	新規アカウントは12ヶ月無料枠、以降は\$3.50/百万回。この規模ならごく僅か
DynamoDB（7テーブル）	読み書きリクエスト数・ストレージ	無料枠+低ボリュームでごく僅か
S3（静的ホスティング+アップロード）	ストレージ・リクエスト数	数百円未満想定
CloudFront	データ転送量	12ヶ月無料枠（1TB/月）、以降も低ボリュームなら僅か
SSM Parameter Store	Standard利用なら無料	無料
EventBridge（日次バッチ起動）	ルール数・イベント数	無料枠内
Cognito【新規】	月間アクティブユーザー数	無料枠（実質1万MAU程度）内、友人数名なら無料
AWS Budgets【新規】	予算数	2つまで無料。今回1つのみ設定
Anthropic Claude API（Haiku、2026-07-26にGeminiから移行）	トークン数（画像添付は増加）	最も課金リスクが高い部分 。無料枠は無く完全従量課金だがHaikuは低単価。AI相談への1日あたりレート制限（DAILY_CHAT_LIMIT）、AI分析・新スポット探索・釣具店検索へのレート制限で抑制済み
Google Places API	リクエスト数	月\$200無料クレジット枠内の想定
Google Maps JavaScript API	読み込み回数	同上、\$200クレジット枠
Cloudflare Workers（ミラー）	リクエスト数	無料枠（1日10万リクエスト）で十分

AWS側は `CostBudget`（AWS Budgets、月\$10・80%/100%到達時にメール通知）で自動監視している。Google Cloud側（Places/Maps）とAnthropic側（Claude）はAWS Budgetsの対象外のため、それぞれのコンソールで別途予算アラート・spend limit

を手動設定する必要がある（README.mdの「10. 利用料アラート」参照）。

7. 関連ドキュメント

- 要件・仕様の背景: [要件定義.html](#)、[企画書.html](#)
- AWS構成・実装更新の経緯: [基本設計書.html](#)
- Lambda単位の詳細仕様: [詳細設計.html](#)
- テーブル設計: [DEテーブル設計.html](#)